



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant Teaching
Professor
Lisa Yan

IEC Prefixes, C Generics



Agenda

IEC prefixes (aka "kibi, mebi, gibi, ...")

- IEC Prefixes
- Pointer Practice
- Generic Functions
- Writing Generics: Episode II
- [notes] swap_ends
- [notes] Function Pointers



Motivation



On a 32-bit architecture,
we have access to **4 GiB** "gibibytes" of memory.

However, on a 64-bit architecture,
we have access to **16 ExB** "exbibytes" of memory!

Huh?



student



SI vs IEC Prefixes

Prefix (Abbr)	SI Size
Kilo (K)	$10^3 = 1,000$
Mega (M)	$10^6 = 1,000,000$
Giga (G)	$10^9 = 1,000,000,000$
Tera (T)	$10^{12} = 1,000,000,000,000$
Peta (P)	$10^{15} = 1,000,000,000,000,000$
Exa (E)	$10^{18} = 1,000,000,000,000,000,000$
Zetta (Z)	$10^{21} = 1,000,000,000,000,000,000,000$
Yotta (Y)	$10^{24} = 1,000,000,000,000,000,000,000,000$

Factor	IEC (Abbr)
$2^{10} = 1,024$	Kibi (Ki)
$2^{20} = 1,048,576$	Mebi (Mi)
$2^{30} = 1,073,741,824$	Gibi (Gi)
$2^{40} = 1,099,511,627,776$	Tebi (Ti)
$2^{50} = 1,125,899,906,842,624$	Pebi (Pi)
$2^{60} = 1,152,921,504,606,846,976$	Exbi (Ei)
$2^{70} = 1,180,591,620,717,411,303,424$	Zebi (Zi)
$2^{80} = 1,208,925,819,614,629,174,706,176$	Yobi (Yi)

Base-10: Common use prefixes

- (SI units, except K [= k in SI])

$$10^3 \approx 2^{10}$$

Base-2: IEC Standard Prefixes

- Binary quantities (bi: "bee")
- Officially up to exbi ("exbee")

Yan, SP26



What units to use?

**JOHN DINAN; et al., Plaintiffs-Appellants,
v.**

SANDISK LLC, Defendant-Appellee.

[No. 20-15287.](#)

United States Court of Appeals, Ninth Circuit.

Argued and Submitted February 4, 2021 — San Francisco, California.
Filed February 10, 2021.

Plaintiffs appeal the district court's Rule 12(b)(6) dismissal of their complaint against SanDisk. Plaintiffs allege that SanDisk violated three California consumer protection statutes^[1] by using gigabyte ("GB") to mean 1,000,000,000 bytes ("the decimal definition"), when Plaintiffs assumed gigabyte as used by SanDisk meant 1,073,741,824 bytes ("the binary definition"). Plaintiffs additionally allege a breach of contract on similar grounds.

$$1 \text{ GB} = (10^3)^3$$

$$1 \text{ GiB} = (2^{10})^3$$

Get familiar with
both base-10 and
base-2 prefixes!



Agenda

Practice with Pointers

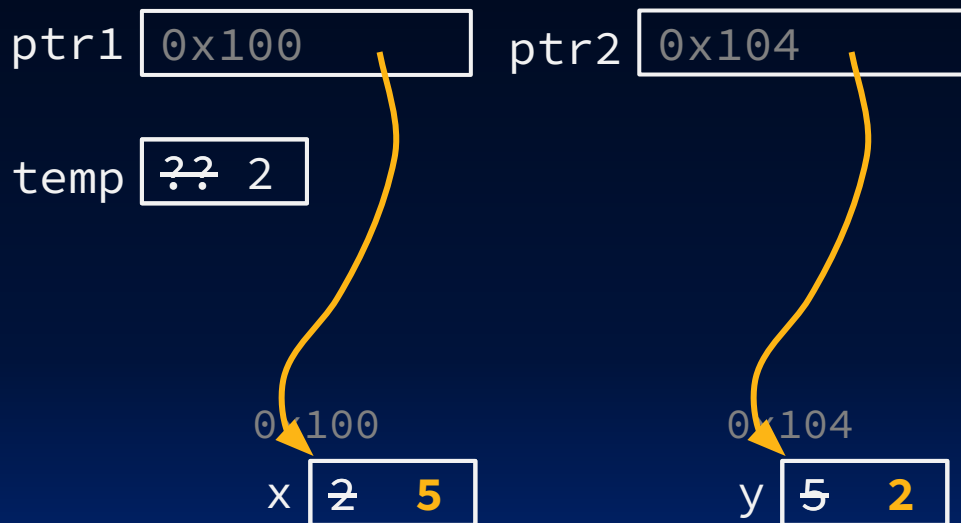
- IEC Prefixes
- Pointer Practice
- Generic Functions
- Writing Generics: Episode II
- [notes] swap_ends
- [notes] Function Pointers

Swap ints

```
1 void swap_int(int *ptr1,  
2               int *ptr2) {  
3     int temp = *ptr1;  
4     *ptr1 = *ptr2;  
5     *ptr2 = temp;  
6 }
```

```
6 int main() {  
7     int x = 2;  
8     int y = 5;  
9     swap_int(&x, &y);  
10    ...  
11    return 0;  
12 }
```

- Remember: C is pass-by-value.
 - Pass in arguments to swap_int as **pointers** to swap local variables x, y in main.



Swap other types

```
1 void swap_int(int *ptr1,  
2               int *ptr2) {  
3     int temp = *ptr1;  
4     *ptr1 = *ptr2;  
5     *ptr2 = temp;  
6 }
```

```
7 void swap_short(short *ptr1,  
8                 short *ptr2) {  
9     short temp = *ptr1;  
10    *ptr1 = *ptr2;  
11    *ptr2 = temp;  
12 }
```

```
13 void swap_string(char **ptr1,  
14                  char **ptr2) {  
15     char *temp = *ptr1;  
16     *ptr1 = *ptr2;  
17     *ptr2 = temp;  
18 }
```

- With different types, the logic is still similar.



Swap "strings" (1/3)

What does
swap_string do?

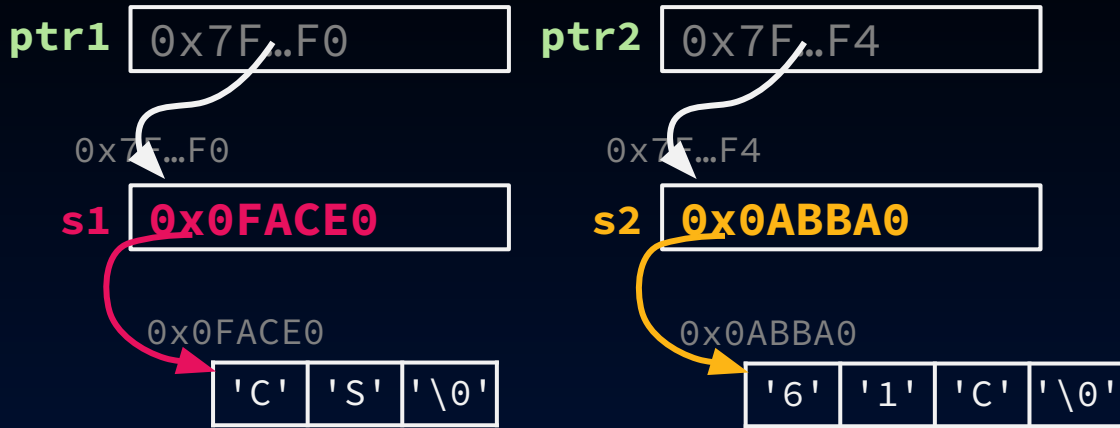


```
11 void swap_string(char **ptr1,  
12                  char **ptr2) {  
13     char *temp = *ptr1;  
14     *ptr1 = *ptr2;  
15     *ptr2 = temp;  
16 }
```

```
int main() {  
    char *s1 = "CS";  
    char *s2 = "61C";  
    swap_string(&s1, &s2);  
    ...  
    return 0;  
}
```



Swap "strings" (2/3)

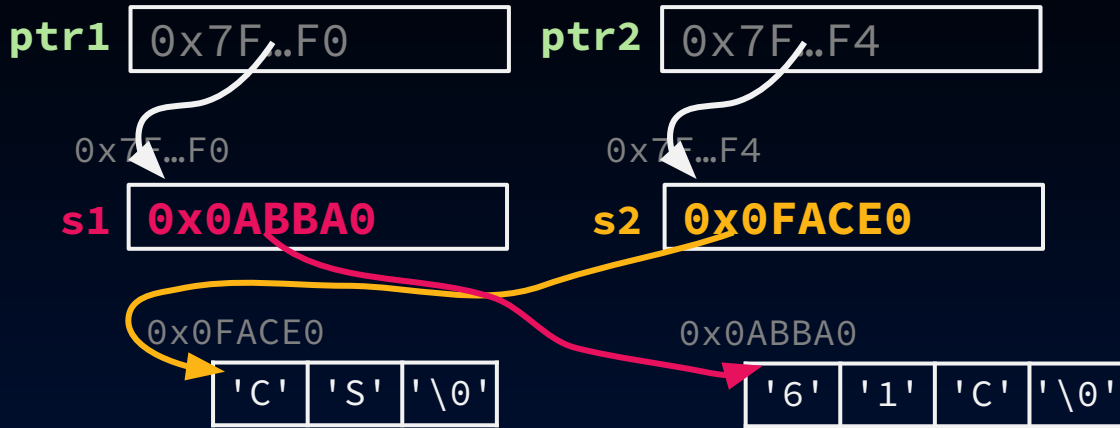


```
11 void swap_string(char **ptr1,  
12                  char **ptr2) {  
13     char *temp = *ptr1;  
14     *ptr1 = *ptr2;  
15     *ptr2 = temp;  
16 }
```

```
int main() {  
    char *s1 = "CS";  
    char *s2 = "61C";  
    swap_string(&s1, &s2);  
    ...  
    return 0;  
}
```



Swap "strings" (3/3)



```
11 void swap_string(char **ptr1,  
12                  char **ptr2) {  
13     char *temp = *ptr1;  
14     *ptr1 = *ptr2;  
15     *ptr2 = temp;  
16 }
```

```
int main() {  
    char *s1 = "CS";  
    char *s2 = "61C";  
    swap_string(&s1, &s2);  
    ...  
    return 0;  
}
```

Generic swap...?

```
1 void swap_int(int *ptr1,
2               int *ptr2) {
3     int temp = *ptr1;
4     *ptr1 = *ptr2;
5     *ptr2 = temp;
6 }
7
8 void swap_short(short *ptr1,
9                 short *ptr2) {
10     short temp = *ptr1;
11     *ptr1 = *ptr2;
12     *ptr2 = temp;
13 }
14
15 void swap_string(char **ptr1,
16                  char **ptr2) {
17     char *temp = *ptr1;
18     *ptr1 = *ptr2;
19     *ptr2 = temp;
20 }
```



```
void swap(void *ptr1,
          void *ptr2) {
    ...
}
```

Can we write this generic function?



Agenda

Generic Functions

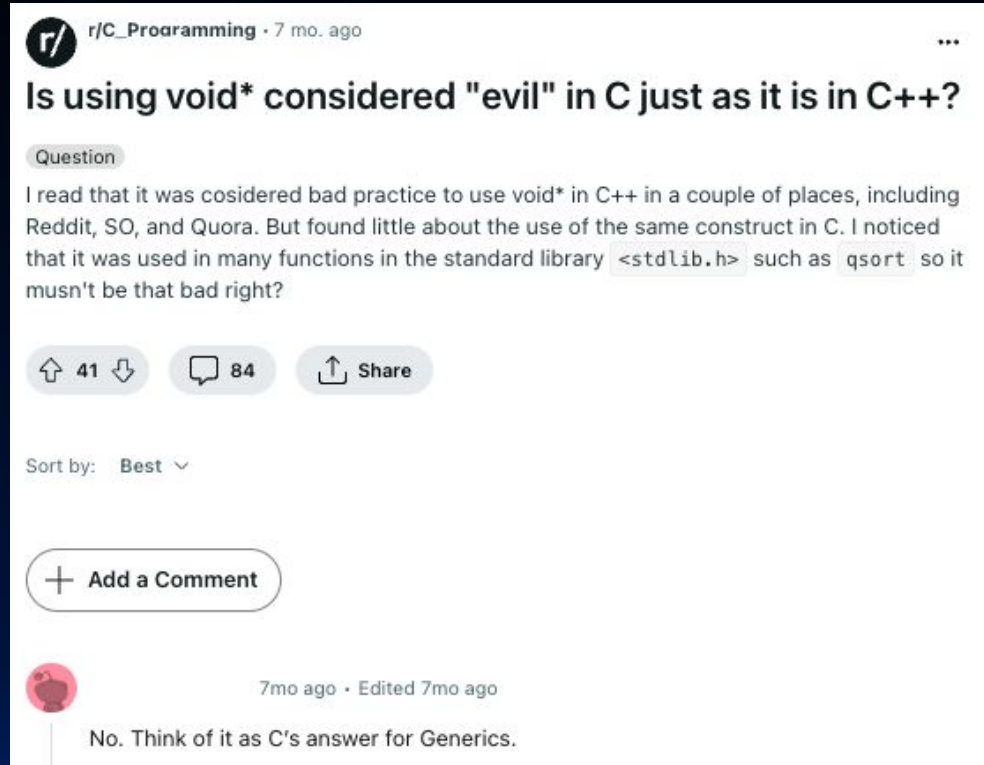
- IEC Prefixes
- Pointer Practice
- Generic Functions
- Writing Generics: Episode II
- [notes] swap_ends
- [notes] Function Pointers



When should we use void *?

- **void *** is a type that can point to anything (**generic pointer**).
 - Use sparingly to help avoid program bugs... and security issues...and more...
- ...but generic pointers have their uses!

Generics is short for **generic functions**.



Yan, SP26



You have seen generics already

- Example generics: the `stdlib.h` heap functions
 - `accept/return generic pointers`

MALLOC(3) Linux Programmer's Manual MALLOC(3)

SYNOPSIS

```
#include <stdlib.h>
```

```
void *malloc(size_t size);
```

```
void free(void *ptr);
```

```
void *calloc(size_t nmem, size_t size);
```

```
void *realloc(void *ptr, size_t size);
```



Why generic functions?

- We want to write general-purpose code.
 - Generic code reduces code duplication and means you can make improvements and fix bugs in one place rather than many.
 - **Generics** are used throughout C for functions to sort any array, search any array, free arbitrary memory, and more.
- In general, generics:
 - Should work regardless of argument type
 - Update blocks of memory
 - Regardless of type of data stored in those blocks

Generic Swap: A first attempt

- Input: double pointer???
- Address
- Cast
- Temp: char array???
- Syntax sucks???

```
void swap_int(int *ptr1,
              int *ptr2) {
    int temp = *ptr1; // 1
    *ptr1 = *ptr2;    // 2
    *ptr2 = temp;     // 3
}
```

```
1 void swap(void *ptr1, void *ptr2) {
2     // 1. store a copy of data1 in temporary storage
3     void temp = *ptr1;
4
5
6     // 2. copy data2 to location of data1
7     *ptr1 = *ptr2;
8
9     // 3. copy data in temporary storage to location of data2
10    *ptr2 = temp;
11 }
```



Yan, SP26

Dereferencing is not a generic operation; it is typed

```
void swap_int(int *ptr1,
              int *ptr2) {
    int temp = *ptr1; // 1
    *ptr1 = *ptr2;    // 2
    *ptr2 = temp;     // 3
}
```

```
1 void swap(void *ptr1, void *ptr2) {
2
3     void temp = *ptr1;
4
5
6
7     *ptr1 = *ptr2;
8
9
10    *ptr2 = temp;
11 }
```

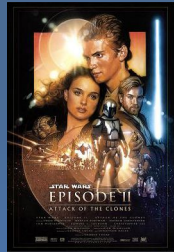
warning: dereferencing '**void ***' pointer
error: invalid use of void expression

To dereference a pointer, we must know the number of bytes to access in memory at **compile time**.

Generic pointers (**void ***) **cannot** use the dereference operator!

Writing Generics: Episode II

A star (*) was
reference



- IEC Prefixes
- Pointer Practice
- Generic Functions
- Writing Generics: Episode II
- [notes] swap_ends
- [notes] Function Pointers

Generic memory copying: memcpy, memmove

- To access some number of bytes in memory using a void * pointer, use memcpy, memmove from the string.h standard library.
 - Both generics **copy** n bytes from memory area **src** to memory area **dest** and return a pointer to dest.

```
void *memcpy(void *dest,
             const void *src, size_t n);
```

man memcpy: "The memory areas **must not overlap**."

```
void *memmove(void *dest,
              const void *src, size_t n);
```

man memmove: "copying takes place as though the bytes in src are first copied into a **temporary array** ... then copied ... to dest."

- For **performance reasons**, use memcpy—unless you know memory areas overlap.
 - Not only is memcpy faster, but some implementations of memmove actually employ temporary storage (like in C99), which risks running out of memory. [\[source1\]](#), [\[source2\]](#)

Intuition: memcpy vs. dereference

```
1 void swap_int(int *ptr1,  
2               int *ptr2) {  
3     int temp = *ptr1;  
4     *ptr1 = *ptr2;  
5     *ptr2 = temp;  
6 }
```



```
void *memcpy(void *dest,  
             const void *src,  
             size_t n);
```

`memcpy(_ptr1, _ptr2, sizeof(int));`

- Let's rewrite Line 3 to call memcpy instead of dereferencing.

To copy bytes between memory locations, we must know **how many bytes** to read/write.



Constructing Generic Swap

Third parameter:
element size



```
1 void swap(void *ptr1, void *ptr2, size_t nbytes) {  
2     // 1. store a copy of data1 in temporary storage  
3  
4  
5  
6     // 2. copy data2 to location of data1  
7  
8  
9     // 3. copy data in temporary storage to location of data2  
10  
11 }
```

Constructing Generic Swap: 3 memcpy calls

```
void swap_int(int *ptr1,
              int *ptr2) {
    int temp = *ptr1; // 1
    *ptr1 = *ptr2;    // 2
    *ptr2 = temp;     // 3
}
```

```
1 void swap(void *ptr1, void *ptr2, size_t nbytes) {
2     // 1. store a copy of data1 in temporary storage
3     ???????????? // allocate space for temp
4     memcpy(temp, ptr1, nbytes);
5
6     // 2. copy data2 to location of data1
7     memcpy(ptr1, ptr2, nbytes);
8
9     // 3. copy data in temporary storage to location of data2
10    memcpy(ptr2, temp, nbytes);
11 }
```

How to declare temp
storage on stack?



Yan, SP26

Constructing Generic Swap: Byte Array

- Suppose:
 - ptr1 points to nbytes of memory (e.g., data1)
 - ptr2 points to nbytes of memory (e.g., data2)
 - Assume no overlap in these two memory areas

```
void swap_int(int *ptr1,
              int *ptr2) {
    int temp = *ptr1; // 1
    *ptr1 = *ptr2;    // 2
    *ptr2 = temp;     // 3
}
```

```
1 void swap(void *ptr1, void *ptr2, size_t nbytes) {
2     // 1. store a copy of data1 in temporary storage
3     char temp[nbytes];
4     memcpy(temp, ptr1, nbytes);
5
6     // 2. copy data2 to location of data1
7     memcpy(ptr1, ptr2, nbytes);
8
9     // 3. copy data in temporary storage to location of data1
10    memcpy(ptr2, temp, nbytes);
11 }
```

In C, `sizeof(char) == 1` byte, regardless of architecture.

To create temporary “generic” storage, declare a local character array (i.e., **buffer**).

If temp is a local **byte array**, this resolves to a pointer.

Generic Swap in Action

```
1 void swap(void *ptr1,
2           void *ptr2,
3           size_t nbytes) {
4     char temp[nbytes];
5     memcpy(temp, ptr1, nbytes);
6     memcpy(ptr1, ptr2, nbytes);
7     memcpy(ptr2, temp, nbytes);
8 }
9
10 int main() {
11     int data1 = 22;
12     int data2 = 61;
13     swap(&data1,
14         &data2,
15         sizeof(data1));
16     return 0;
17 }
```

ptr1 0x100

ptr2 0x104

temp ?? ?? ?? ??

0x100
data1 22

0x104
data2 61

(assume sizeof(int) == 4)

Generic Swap in Action

```
1 void swap(void *ptr1,
2           void *ptr2,
3           size_t nbytes) {
4     char temp[nbytes];
5     memcpy(temp, ptr1, nbytes);
6     memcpy(ptr1, ptr2, nbytes);
7     memcpy(ptr2, temp, nbytes);
8 }
9
10 int main() {
11     int data1 = 22;
12     int data2 = 61;
13     swap(&data1,
14         &data2,
15         sizeof(data1));
16     return 0;
17 }
```

(assume sizeof(int) == 4)

ptr1 0x100

ptr2 0x104

temp 22

0x100

data1 61

0x104

data2 22



And in Conclusion...

- Function pointers enable higher-order functions in C.
 - `map`, `filter`, `sorting`, etc.
- Generic functions (i.e., generics), use `void *` pointers to operate on memory.
 - Generics are widely present in the C standard library! (`malloc`, ...)
 - Generics require a solid understanding of memory; by manipulating arbitrary bytes, you risk violating data boundaries, e.g., **"Frankenstein"**-ing two halves of ints.
- When writing generics:
 - Generic pointers do not support dereferencing. Instead, use byte handling functions (`memcpy`, `memmove`).
 - Pointer arithmetic: first cast to byte arrays with `(char *)`.





Agenda

More Practice in Course Notes

- IEC Prefixes
- Pointer Practice
- Generic Functions
- Writing Generics: Episode II
- [notes] swap_ends
- [notes] Function Pointers